

DÉPARTEMENT INFORMATIQUE - IUT2 GRENOBLE



Année Universitaire 2024-2025
RAPPORT SAÉ 4.01

RAPPORT FINAL DE SAE 4.01

Groupe 15

Présenté par

Paul Cogitore, Cyrian Torrejon, Moras Evan, Tsigris Nicolas

Sommaire

Sommaire	2
Introduction	3
.1 Contexte et objectifs du projet	3
Présentation de l'application existante et de son rôle	3
Enjeux et objectifs de l'amélioration (qualité, ergonomie, performance)	3
Cadre et organisation du travail en équipe	3
.2 Méthodologie de travail	4
Répartition des tâches et rôles des membres	4
Approche adoptée pour l'analyse et l'amélioration	4
Outils et technologies utilisés	4
I. Rétroconception analyse de l'existant	6
I.1 BackEnd	6
Structure de la base de données	6
Mobile	9
Serveur web Symfony	9
I.2 FrontEnd	9
Mobile	9
Interface web (administrateur, étudiants exceptionnel)	10
II. Améliorations apportées	11
II.1 Améliorations relatives à la qualité du processus de développement (tests, documentation)	11
Niveau BackEnd	11
Niveau FrontEnd	11
II.2 Amélioration et ajouts aux différentes parties de l'application	11
Niveau BackEnd	11
Niveau FrontEnd	12
III. Résultats et évaluation du projet	15
III.1 Synthèse des améliorations mises en place	15
Bilan des optimisations et de leur impact	15
Structure de la base de données	15
Comparaison avant/après	15
III.2 Retour d'expérience et perspectives	15
Difficultés rencontrées et solutions mises en œuvre	15
Axes d'amélioration pour une future version	15
IV. Annexes	16
IV. X ANNEXE X : Annexe Docker	16
IV. X ANNEXE X : Images interfaces android studio	20
Image MainActivity	20
Image listOffres	20
Image EditCandidature	20
IV. X ANNEXE X : Références webographiques	22

Introduction

.1 Contexte et objectifs du projet

Présentation de l'application existante et de son rôle

L'application est une plateforme dédiée à la gestion de la recherche de stage des étudiants de l'IUT2 de Grenoble et à l'administration des offres par les responsables des stages. Elle se compose d'une application mobile permettant aux étudiants de consulter les offres, marquer celles qui les intéressent, déclarer leurs candidatures et en suivre l'évolution jusqu'à la validation d'une proposition acceptée. L'application intègre également des rappels en cas d'absence de réponse et permet de mettre à jour le statut des candidatures après un entretien. Parallèlement, une application web est mise à disposition des responsables des stages pour gérer les offres et suivre les candidatures des étudiants via un tableau de bord. Le tout repose sur un service web central assurant l'authentification et la gestion des actions, ainsi qu'une base de données hébergée sur un serveur Debian pour garantir la persistance des informations. Cette solution numérique simplifie et optimise l'ensemble du processus de recherche de stage, du premier contact avec les offres jusqu'à la sélection finale d'une proposition.

Enjeux et objectifs de l'amélioration (qualité, ergonomie, performance)

L'amélioration de cette application de gestion de stages présente des enjeux stratégiques majeurs pour optimiser l'expérience des étudiants et responsables tout en renforçant la robustesse du système. Sur le plan de la qualité, il s'agit d'assurer une fiabilité accrue des échanges de données entre les quatre modules, de garantir l'intégrité des informations stockées localement et sur le serveur, et d'implémenter un système de notification plus précis pour les relances. L'ergonomie constitue un levier essentiel pour faciliter l'adoption de l'outil, notamment en simplifiant la navigation entre les différentes phases du processus de recherche, en rendant plus intuitif le changement d'état des candidatures, et en proposant des interfaces adaptées aux usages spécifiques des étudiants (mobile) et des responsables (web). Quant aux performances, l'optimisation de la synchronisation des données entre l'application mobile et le serveur, la réduction des temps de réponse lors des consultations d'offres, et l'amélioration de la gestion des ressources sur la machine virtuelle Debian permettront de supporter efficacement une montée en charge lors des périodes de pointe de recherche de stages, tout en assurant une expérience fluide et réactive pour l'ensemble des utilisateurs.

Cadre et organisation du travail en équipe

Afin de maintenir une coordination optimale, nous avons mis en place des outils adaptés à nos besoins. Git a été utilisé pour gérer le code source et garantir une version contrôlée du projet. Pour la communication quotidienne, Discord a été notre principal outil, nous permettant d'échanger rapidement et de nous réunir régulièrement en visioconférence pour discuter de l'avancement du projet. Google Docs a facilité la rédaction collaborative des

documents, tandis que Notion a été utilisé pour organiser et suivre les tâches à réaliser, ainsi que pour centraliser les informations importantes du projet. Cette organisation a permis une bonne gestion de notre travail à distance, en favorisant une collaboration fluide et en assurant le respect des délais.

.2 Méthodologie de travail

Répartition des tâches et rôles des membres

L'organisation du travail en équipe a été rigoureusement planifiée avant le sprint, avec une répartition des tâches selon les compétences individuelles et les priorités du projet. La structure initiale prévoyait une distribution équilibrée : un membre pour la conteneurisation Docker, un autre pour le développement Symfony, deux pour l'application Android, et un dernier pour les tests. Malgré une cohésion d'équipe solide, l'absence d'un membre a perturbé le planning, forçant l'équipe à quatre à réorganiser la distribution des charges. Cette situation a exigé une augmentation des contributions individuelles, notamment l'intégration des tests dans les responsabilités des autres membres, entraînant un débordement sur leur temps personnel. La méthodologie de travail a été ajustée avec une coordination quotidienne renforcée et une priorisation des tâches critiques. La journée finale a permis la consolidation des travaux, incluant la rédaction du rapport et la validation des applications, livrant un ensemble fonctionnel conforme aux spécifications malgré les contraintes.

Approche adoptée pour l'analyse et l'amélioration

Pour l'amélioration, nous avons opté pour une approche itérative, mettant en œuvre des changements progressifs. Par exemple, nous avons d'abord optimisé certaines fonctionnalités clés du code avant de repenser l'interface utilisateur pour la rendre plus conviviale. Nous avons ensuite testé les nouvelles fonctionnalités avec un groupe d'utilisateurs pour valider les améliorations et recueillir des retours. Ces retours nous ont permis d'ajuster certaines modifications avant de les déployer.

Outils et technologies utilisés

Au cours du développement, nous avons utilisé plusieurs outils pour améliorer les différents composants de l'application. Pour l'application Android, nous avons utilisé Figma pour améliorer et tester le visuel des interfaces et Android Studio pour coder l'interface choisie et corriger les problèmes dans le code préexistant. Nous avons aussi utilisé l'outil PhpStorm pour améliorer tout ce qui concerne l'application Web car l'application utilisait le framework Symfony qui est supporté par PhpStorm. Nous avons utilisé VirtualBox et Qemu pour créer les machines virtuelles qui allaient contenir le serveur Web et la base de données et Docker pour les déployer. Enfin nous avons utilisé Gricad-Gitlab comme logiciel de gestion de versions de code car un dépôt nous avait déjà attribué par la maîtrise d'ouvrage pour nous aider. Aussi, en plus des raisons citées ci-dessus, ces outils ont été choisis car ils nous sont familiers et/ou nous ont été recommandé par la maîtrise d'ouvrage.

I. Rétroconception analyse de l'existant

I.1 BackEnd

Structure de la base de données

Analyse des Problèmes de Données

Certains problèmes peuvent être détectés dans la conception actuelle :

Redondance de stockage des états.

`etat_candidature`, `etat_offre`, et `etat_recherche` stockent des états sous forme de texte (`etat + descriptif`). Idéalement, ces états devraient être utilisés avec des valeurs fixes en ENUM ou dans une table séparée sans descriptif, pour éviter la duplication des valeurs.

Stockage des mots-clés (`mots_cles` dans `offre`) sous forme de chaîne.

Actuellement stocké en `VARCHAR(255)`, ce qui limite la recherche et la normalisation. Il faudrait une table `mot_cle` et une table d'association `offre_mot_cle` pour gérer proprement cette relation (car une offre peut avoir plusieurs mots-clés et inversement).

Les rôles (`roles` dans `compte_etudiant`) stockés en JSON.

Cela complexifie les requêtes SQL. Une table séparée `compte_etudiant_role` avec une relation N-N entre `compte_etudiant` et `role` serait plus adaptée.

Problèmes d'intégrité référentielle et contraintes :

Champ `parcours` (dans `compte_etudiant` et `offre`) mal défini.

Il s'agit d'un `VARCHAR(1)`, mais aucune contrainte n'est spécifiée pour limiter les valeurs possibles. Une table `parcours` avec des valeurs prédéfinies serait préférable.

Manque d'unicité dans certaines relations.

`candidature` pourrait imposer une contrainte pour empêcher un étudiant de postuler plusieurs fois à la même offre. `offre_consultee` et `offre_retenue` devraient imposer une clé composite (`compte_etudiant_id`, `offre_id`) pour éviter les doublons.

Deux tables ne font pas partie de la structure métier initialement prévue de la base de données, mais a été générée automatiquement par un composant Symfony à des fins techniques. `notify_messenger_messages` et `doctrine_migration_versions` qui sont issus de l'impact du web sur la structure de la base. La première est utilisée pour gérer des messages asynchrones (comme l'envoi d'e-mails, de notifications, ou d'autres traitements en arrière-plan) tandis que la deuxième assure un suivi fiable des modifications de la base de données et facilite le déploiement sur différents environnements.

Analyse des Dépendances Fonctionnelles

Une dépendance fonctionnelle (DF) est une contrainte entre deux ensembles d'attributs :

$A \rightarrow B$ signifie que si deux lignes ont la même valeur pour A, alors elles ont forcément la même valeur pour B.

Principales dépendances dans les tables :

Table `candidature`

`id` $\rightarrow$ (`compte_etudiant_id`, `offre_id`, `etat_candidature_id`, `type_action`, `date_action`) (L'identifiant de la candidature détermine toutes les autres valeurs).
`(compte_etudiant_id, offre_id)` $\rightarrow$ `etat_candidature_id` (Un étudiant ne peut avoir qu'un seul statut par offre).

Table `offre`

`id` $\rightarrow$ (`etat_offre_id`, `entreprise_id`, `intitule`, `descriptif`, `date_depot`, `parcours`, `mots_cles`, `url_piece_jointe`) (L'identifiant d'une offre détermine toutes ses propriétés).
`etat_offre_id` $\rightarrow$ `etat` (Etat d'une offre dépend uniquement de `etat_offre_id`).

Table `compte_etudiant`

`id` $\rightarrow$ (`etat_recherche_id`, `login`, `roles`, `password`, `parcours`, `derniere_connexion`, `etudiant_id`) (Un compte étudiant a une seule identité).
`etudiant_id` $\rightarrow$ `id` (Un étudiant ne peut avoir qu'un seul compte).

Normalisation des Tables

Analysons maintenant les formes normales (FN) de chaque table.

Première forme normale (1FN) :

Une table est en 1FN si :

- Toutes les valeurs sont atomiques (pas de colonnes multi-valeurs ou de tableaux).
- Chaque ligne a une clé primaire unique.

Ici toutes les tables respectent la 1FN à l'exception de `mots_cles` et `roles` qui sont stockés sous forme de texte ou JSON, donc elles devraient être séparées.

Deuxième forme normale (2FN) :

Une table est en 2FN si :

- Elle est en 1FN.
- Chaque colonne dépend entièrement de la clé primaire (pas de dépendance partielle dans les clés composites).

Problèmes détectés dans la base actuelle

Problème dans `candidature`

- `(compte_etudiant_id, offre_id) → etat_candidature_id` montre qu'`etat_candidature_id` ne dépend pas uniquement de `id`, mais aussi de `(compte_etudiant_id, offre_id)`.
- Solution : Déplacer `etat_candidature_id` dans une table `historique_candidature`.

Problème dans `offre`

- `etat_offre_id → etat, descriptif`, ces valeurs sont dépendantes de `etat_offre_id`, donc elles devraient être stockées séparément et accédées via une relation.

Problème dans `compte_etudian`

- `etat_recherche_id → etat, descriptif`, même problème que précédemment, à normaliser dans `etat_recherche`.

Troisième forme normale (3FN) :

Une table est en 3FN si :

- Elle est en 2FN.
- Aucune colonne ne dépend d'une autre colonne non-clé.

Problème dans `etat_candidature` et `etat_offre`

- `descriptif` dépend de `etat` et non de l'ID.
- Solution : Laisser `etat` en tant que clé primaire et supprimer `descriptif` ou le mettre dans une autre table.

Problème dans `offre`

- `mots_cles` est un champ dépendant de `offre.id`, mais il contient plusieurs valeurs séparées.
- Solution : Créer une table `mot_cle` et une table de jointure `offre_mot_cle`.

Voici la liste de toutes les dépendances fonctionnelles

- `candidature.id → compte_etudiant_id, offre_id, etat_candidature_id, type_action, date_action`
- `compte_etudiant.id → etat_recherche_id, login, roles, password, parcours, derniere_connexion, etudiant_id`
- `login → etat_recherche_id, roles, password, parcours, derniere_connexion, etudiant_id`
- `entreprise.id → raison_sociale, ville, pays`
- `etat_candidature.id → etat, descriptif`
- `etat_offre_id → etat, descriptif`
- `etat_recherche.id → etat, descriptif`
- `etudiant.id → numero_ine, nom, prenom, email`

- numero_in → nom, prenom, email
- offre.id → etat_offre_id, entreprise_id, intitule, descriptif, date_depot, parcours, mots_cles, url_piece_jointe
- offre_consultee.id → compte_etudiant_id, offre_id
- offre_retenue.id → compte_etudiant_id, offre_id

Mobile

L'architecture du backend mobile présentait des déficiences majeures concernant principalement les appels à l'API, caractérisés par une implémentation peu optimisée et problématique sur plusieurs aspects techniques. Ces appels souffraient d'une importante redondance, multipliant inutilement les échanges avec le serveur pour des données similaires ou déjà disponibles. Le code existant générait également des fuites de mémoire substantielles dues à une gestion inadéquate des connexions et des ressources système. Cette configuration technique défailante exposait l'infrastructure à des risques potentiels de surcharge de la base de données, particulièrement lors des périodes d'utilisation intensive, ou en autorisant des données inutiles comme des doublons compromettant ainsi la stabilité et la réactivité globales du système.

Serveur web Symfony

Lors de l'analyse approfondie du serveur Symfony fournit, nous avons identifié plusieurs points nécessitant des ajustements pour améliorer la qualité et l'organisation du code :

- Certaines parties de certains contrôleurs contenaient des fonctions ou méthodes redondantes. Bien que fonctionnelles, ces répétitions devraient idéalement être isolées dans des services dédiés pour accroître la modularité, simplifier la maintenance et améliorer la lisibilité globale du code.
- Concernant la fonctionnalité "Rester connecté" ou "Remember me", sa mise en œuvre nécessite plus que le simple décommentaire d'une section du fichier `login.html.twig`. Une configuration complémentaire et des ajustements techniques sont requis pour garantir son bon fonctionnement. Ce point sera traité ultérieurement pour une intégration optimale.
- Certaines fonctions présentaient des problèmes d'optimisation, compromettant potentiellement la performance globale du serveur. Nous avons pris des mesures pour résoudre ces dysfonctionnements en procédant à des ajustements techniques ciblés.

I.2 FrontEnd

Mobile

L'interface utilisateur préexistante de l'application mobile souffrait de carences ergonomiques significatives compromettant l'expérience utilisateur. La disposition des éléments graphiques manquait de cohérence structurelle, avec un placement apparemment arbitraire des

composants d'interaction qui nuisait à la compréhension intuitive du flux de navigation. Cette conception déficiente se manifestait également par l'absence de points d'entrée critiques vers certaines fonctionnalités essentielles, rendant plusieurs capacités du système inaccessibles en raison d'un manque de boutons ou de contrôles visibles. Au-delà de ces problèmes d'accessibilité, l'interface contenait des portions de code non exploitées, maintenant des fonctionnalités résiduelles sans utilité réelle pour les utilisateurs finaux. La fiabilité technique était également compromise par des systèmes de compteurs défectueux dont les mécanismes d'incrémentation incorrects généreraient des données erronées, affectant potentiellement l'intégrité des informations présentées lors du suivi des candidatures.

Interface web (administrateur, étudiants exceptionnel)

Notre analyse a également révélé des lacunes notables dans l'organisation et l'esthétique générale de l'interface utilisateur :

- L'agencement des boutons s'est avéré perfectible. Beaucoup de boutons se présentaient sous la forme de liens, ce qui manquait de clarté et nuit à l'ergonomie.
- Le recours à un menu déroulant unique regroupant un grand nombre d'éléments rendait la navigation peu intuitive et réduisait l'agrément d'utilisation de l'application.
- Ces problématiques mettent en évidence la nécessité d'une refonte visuelle et structurelle pour améliorer l'expérience utilisateur et l'intuitivité de l'interface.

II. Améliorations apportées

II.1 Améliorations relatives à la qualité du processus de développement (tests, documentation)

Niveau BackEnd

Dans notre projet, nous avons mis en place des tests unitaires qui couvrent toutes les entités présentes dans le projet ainsi que les services. Cela nous permet donc de confirmer leur fonctionnement correct. Nous n'avons pas pu mettre en place les tests d'interface en raison du manque de travail de l'un de nos camarades ce qui nous a considérablement handicapé. En effet, en raison de cet incident nous avons fait le choix de faire le plus possible tout en essayant de ne rien laisser à part.

Un fichier Readme a été écrit pour l'aide à la mise en place des conteneurs Docker, il contient la procédure pour réaliser l'installation du serveur.

Niveau FrontEnd

Concernant le FrontEnd, une attention particulière a été portée à la qualité de l'interface utilisateur, notamment en mettant en œuvre des tests automatisés d'interface mobile. Ces tests ont permis de vérifier l'exactitude de l'application de différentes fonctionnalités.

II.2 Amélioration et ajouts aux différentes parties de l'application

Niveau BackEnd

Pour la partie Docker : La conteneurisation du service web s'appuie sur deux images Docker stratégiquement sélectionnées : php:8.2-apache pour l'application Symfony, qui contient PHP 8.2, un serveur web intégré Apache ainsi que les extensions nécessaires (comme pdo, pdo_pgsql, intl, etc.) (l'image php:8.3-apache ayant généré des erreurs dues à du code déprécié). Et une de base de données PostgreSQL avec l'image postgres:latest, dernière version stable de postgres.

L'image Symfony a été enrichie avec Git (pour automatiser la récupération des fichiers depuis Gricad-Gitlab), des dépendances via Composer, et une configuration automatique du serveur Symfony, tandis que l'image PostgreSQL a été augmentée d'un script de création de rôles (create_role.sql) et d'importation d'une sauvegarde initiale via le fichier dumb_BD.sql. L'architecture Docker résultante présente une composition modulaire de 42 couches pour Symfony et 24 couches pour PostgreSQL, chaque ajout créant une nouvelle couche pour maintenir la réutilisabilité. Côté configuration, si l'image Symfony n'utilise pas de variables d'environnement spécifiques mais s'appuie sur le fichier .env pour définir des paramètres comme DATABASE_URL, l'image PostgreSQL exploite les variables POSTGRES_PASSWORD=app-stages et POSTGRES_DB=app-stages pour initialiser la base de données. Les commandes de lancement des conteneurs, détaillées en annexe, assurent

le démarrage approprié des services, leur interconnexion via un réseau Docker commun et l'exposition des ports nécessaires pour permettre l'accès aux différentes fonctionnalités.

Optimisation des requêtes API et des fonctions de l'application mobile

La restructuration technique du backend a ciblé les faiblesses structurelles identifiées dans l'architecture des communications avec l'API. La refonte des requêtes réseau a éliminé les redondances critiques qui caractérisent l'implémentation précédente, remplaçant les appels dupliqués par un système de gestion centralisée des requêtes. L'audit des dépendances a permis d'identifier et de supprimer plusieurs bibliothèques superflues qui alourdissent inutilement l'empreinte mémoire de l'application et complexifient la maintenance du code. Cette optimisation stratégique a considérablement amélioré les performances des échanges avec la base de données, réduisant significativement les temps de latence lors des opérations de consultation et de mise à jour des informations relatives aux stages. La nouvelle architecture de communication implique également des mécanismes robustes de gestion des erreurs et de récupération après échec, minimisant ainsi le risque de perte de données ou d'incohérence lors des transferts d'information entre l'application mobile et le serveur central.

Réduction des redondances dans les contrôleurs

Après avoir détecté certaines redondances dans certains contrôleurs notamment les contrôleurs ("EtatOffreContrôleur" et "EtatRechercheContrôleur") nous avons fait le choix de régler ce problème par la création de services qui ont pour but de ne pas avoir de parties de code qui se répètent mais simplement un appel à une fonction similaire. Dans le but de résoudre ce problème nous avons créé un service "EntityService.php" qui a pour objectif de répondre aux besoins de ces deux contrôleurs et de supprimer la redondance de code.

Nous avons trouvé un cas similaire avec les deux contrôleurs ("Admin/TableauDeBordController" et "Etudiant/TableauDeBordController") nous avons donc également créé un autre service dans le but de supprimer cette redondance.

Optimisation des performances des fonctions

Certaines fonctions étaient source d'inefficacité, compromettant les performances globales du serveur. Nous avons analysé ces fonctions en détail, identifié les points de blocage et effectué les modifications nécessaires pour optimiser leur exécution. Ces ajustements incluent la simplification des algorithmes, l'amélioration de la gestion des ressources et la réduction des temps de traitement. Ces efforts garantissent désormais un fonctionnement plus rapide et stable.

Notre projet a bénéficié d'une amélioration majeure grâce à la refonte du schéma de la base de données, que nous avons restructurée selon les principes de la troisième forme normale (Annexe B). Cette normalisation a permis d'éliminer les redondances et les dépendances problématiques, optimisant ainsi l'intégrité des données et les performances du système tout en facilitant la maintenance future de l'application.

Niveau FrontEnd

L'application mobile Android a bénéficié d'une refonte complète de ses interfaces graphiques, apportant une amélioration significative à l'expérience utilisateur. La version précédente présentait de nombreuses lacunes ergonomiques qui rendaient la navigation particulièrement difficile et peu intuitive. Les interfaces étaient désorganisées, avec des éléments disposés sans logique apparente sur chaque page : des boutons insérés aléatoirement au milieu des textes, une densité excessive d'informations rendant la lecture laborieuse, et une absence de cohérence visuelle entre les différentes sections. Les utilisateurs devaient souvent effectuer de nombreuses manipulations pour accéder à des fonctionnalités pourtant essentielles, ce qui générait frustration et perte de temps. L'agencement chaotique des composants et l'absence de hiérarchie visuelle claire compliquaient considérablement l'identification des actions principales disponibles. Cette modernisation s'est articulée autour de plusieurs critères ergonomiques de Bastien et Scapin pour remédier à ces problèmes fondamentaux.

Concernant le critère de Guidage, nous avons amélioré la lisibilité en restructurant toutes les activités avec une disposition plus cohérente des éléments. Dans MainActivity (voir Annexe C: Image MainActivity), les rectangles représentant les offres et candidatures sont désormais identiques et centrés sur l'écran, contrairement à leur disposition aléatoire précédente. Pour le feedback immédiat, les offres consultées apparaissent maintenant grisées en temps réel dans la liste, permettant à l'étudiant d'identifier rapidement celles qu'il a déjà visualisées, évitant ainsi des navigations inutiles (voir Annexe C: Image listOffres).

Pour le critère de Signifiante des codes et dénominations, nous avons implémenté des icônes pertinentes pour représenter les offres et candidatures sur MainActivity. Les codes couleurs ont également été revus, notamment dans l'interface de modification d'une candidature où le bouton de suppression est désormais rouge, renforçant sa fonction d'alerte par rapport au bouton standard utilisé précédemment (voir Annexe C: Image EditCandidature).

En ce qui concerne la Concision, nous avons réduit le nombre d'interactions nécessaires en permettant un accès direct aux candidatures depuis MainActivity, éliminant le parcours obligatoire via la section offres qui rallongent inutilement le processus de consultation.

La Gestion des erreurs a également été améliorée avec le remplacement des messages génériques par des indications précises. Par exemple, lors de la connexion, l'application distingue maintenant les erreurs de mot de passe ou de login à la place d'un message générique, qui donc nous permet de différencier ce genre de souci et des problèmes de réseau (voir Annexe C: Image LoginActivity). Des notifications contextuelles ont été intégrées pour le chargement des listes d'offres ou de candidatures, permettant une meilleure identification des problèmes.

Révision et optimisation de l'organisation visuelle

Afin d'améliorer l'expérience utilisateur, nous avons entièrement revu l'agencement des éléments de l'interface. Les boutons sous forme de liens ont été remplacés par de véritables

boutons interactifs, plus intuitifs et visuellement cohérents. Nous avons également restructuré le menu déroulant, en fragmentant les éléments sur plusieurs menus, afin de simplifier la navigation et de rendre l'utilisation plus agréable. Ces ajustements contribuent à une interface beaucoup plus ergonomique. Nous avons également fait en sorte de revoir l'ensemble des interface visuelle du BackOffice avec l'extension Wave nous avons vérifié les contraste ainsi que la capacité des utilisateur utilisant un descriptif d'écrans d'utiliser l'interface sans problèmes grâce à l'ajout d'élément "Aria-Label" qui ont pour objectif de décrire l'action qu'un certain élément de l'interface vas réaliser.

Implémentation et optimisation de la fonctionnalité "Remember me"

Pour activer la fonctionnalité "Rester connecté", nous avons corrigé les lacunes identifiées. Outre le décommentaire de la section correspondante dans le fichier login.html.twig, des configurations supplémentaires ont été mises en place au niveau de la gestion des sessions et de l'authentification. Nous avons également veillé à intégrer cette fonctionnalité de manière sécurisée, conformément aux bonnes pratiques, pour garantir une expérience utilisateur fluide et fiable.

III. Résultats et évaluation du projet

III.1 *Synthèse des améliorations mises en place*

Bilan des optimisations et de leur impact

L'implémentation de Docker pour un serveur d'application, avec un conteneur dédié à la base de données et un autre à Symfony, offre des avantages considérables par rapport à l'architecture traditionnelle où la BD et Symfony étaient séparés. La maintenabilité s'améliore significativement grâce à l'isolation des environnements, permettant des mises à jour indépendantes sans risque d'effets secondaires entre les composants. Le déploiement devient remarquablement simplifié : au lieu de configurer manuellement chaque serveur, une simple commande Docker suffit pour déployer l'ensemble de l'application avec ses dépendances préconfigurées, garantissant une cohérence parfaite entre les environnements de développement, test et production. Cette approche élimine le problème classique du "ça marche sur ma machine" puisque les conteneurs embarquent toutes leurs dépendances. La scalabilité s'en trouve également optimisée, facilitant la réplication des conteneurs selon les besoins de charge. Enfin, la gestion des versions devient plus transparente, avec la possibilité de revenir rapidement à une version antérieure en cas de problème, réduisant considérablement les temps d'arrêt et améliorant la résilience globale du système.

Les améliorations apportées à l'application mobile s'articulent autour d'une refonte complète tant au niveau de l'interface utilisateur que de l'architecture technique sous-jacente. Visuellement, l'application bénéficie désormais d'une interface redessinée privilégiant l'ergonomie, la lisibilité et l'intuitivité, avec un système de codage couleur permettant d'identifier rapidement l'état des candidatures et des offres. Cette restructuration graphique assure un accès aux fonctionnalités en un minimum d'actions, fluidifiant considérablement l'expérience utilisateur. Sur le plan technique, la couche de communication avec l'API a été repensée pour éliminer les redondances, optimiser les échanges de données et supprimer les bibliothèques superflues qui alourdissaient l'application. Ces améliorations ont permis de résoudre les problèmes de fuites mémoire, de réduire drastiquement les temps de chargement et de renforcer la stabilité des communications avec le serveur. Le résultat est une application plus performante, plus fiable et significativement plus agréable à utiliser, offrant aux étudiants un outil véritablement efficace pour leur recherche de stage.

Pour ce qui est de l'optimisation de l'interface Symfony nous avons comme précédemment évoqué transformé le contenu de certains contrôleurs en services ce qui permet de charger une seule fois le code relatif à certaines fonctions.

Nous avons également fait en sorte que les requêtes SQL qui interrogent la base de données sélectionnent seulement les éléments utilisés et non l'ensemble d'une table ce qui permet encore une fois, de charger moins d'éléments ce qui améliore l'expérience utilisateur.

Avec la nouvelle structure de base de données (Annexe X), toutes les tables sont en troisième forme normale, avec une conversation des dépendances fonctionnelles, ce qui garantit la qualité de la base de données.

Comparaison avant/après

III.2 Retour d'expérience et perspectives

Difficultés rencontrées et solutions mises en œuvre

La principale difficulté rencontrée au cours du projet a été l'absence d'un membre clé de notre équipe pendant une période prolongée. Cette situation a créé un déséquilibre dans la répartition des tâches et a ralenti certains aspects du développement, notamment les tests et la mise en œuvre de certaines fonctionnalités. Pour compenser ce manque de ressources, nous avons réajusté la répartition des tâches entre les membres restants, en nous assurant de prioriser les aspects critiques du projet.

L'implémentation de l'environnement de développement dynamique du backoffice s'est heurtée à d'importants obstacles techniques, principalement liés à la connexion avec la machine virtuelle. Malgré nos multiples tentatives, nous n'avons pas réussi à établir une communication stable entre nos environnements de développement. Cette situation bloquante nous a contraints à faire appel à un professeur dont l'expertise technique a permis de résoudre les problèmes de configuration et de finaliser la mise en place de notre environnement de travail.

Le développement de l'application mobile a été confronté à plusieurs défis techniques qui ont influencé le rythme des optimisations backend. La structure complexe de la base de données existante, combinée à des incohérences terminologiques et des erreurs orthographiques dans les schémas, représente un obstacle initial significatif. L'intégration des communications API avec Java sous Android a constitué une courbe d'apprentissage importante, nécessitant une période d'adaptation et d'expérimentation pour maîtriser cette technologie relativement nouvelle pour certains membres de l'équipe. Les contraintes environnementales, notamment la configuration de machines virtuelles sur les équipements personnels des développeurs, ont également posé des difficultés techniques initialement, bien que ces problèmes aient été résolus efficacement grâce à une collaboration proactive. En définitive, les véritables limitations rencontrées lors de ce projet se sont davantage situées au niveau des ressources disponibles que des obstacles techniques eux-mêmes, la réduction imprévue de l'effectif de l'équipe et les contraintes temporelles ayant constitué les facteurs les plus limitants pour atteindre l'ensemble des objectifs d'optimisation initialement envisagés.

Axes d'amélioration pour une future version

Dans la perspective des axes d'amélioration pour une future version, l'application mobile pourrait bénéficier d'une direction artistique plus audacieuse et originale, intégrant des éléments dynamiques et des animations judicieuses pour insuffler davantage de vie à l'interface utilisateur. L'expérience pourrait être enrichie par l'implémentation de boîtes de dialogue de confirmation contextuelles, renforçant ainsi la prévention des erreurs de manipulation et améliorant la confiance des utilisateurs lors des interactions critiques. Sur le plan technique, une refactorisation plus approfondie du code s'avérerait bénéfique, notamment par l'ajout systématique de commentaires explicatifs facilitant la maintenance future, ainsi que par l'élimination complète des fragments de code vestigiaux qui subsistent encore dans certains modules. L'optimisation des performances pourrait être poursuivie au niveau du backend, en perfectionnant davantage l'architecture des requêtes API à travers une meilleure gestion des caches, et en révisant les commentaires des fonctions pour une cohérence accrue, contribuant ainsi à l'établissement d'une base technique solide et évolutive pour les développements ultérieurs.

Suite à ce projet nous pouvons imaginer plusieurs potentielles améliorations futures telle que la mise en place de statistiques sur les étudiant en recherche de stage, fonctionnalités qui a commencé à être développé mais que nous n'avons pas eu le temps de terminer. D'autre part, nous pourrions améliorer la fonction d'importation d'offres à partir d'un fichier PDF pour pouvoir également traiter des fichiers TXT, ou encore mettre en place un traitement avec de l'intelligence artificielle pour que cette fonction soit plus polyvalente.

Nous pourrions également faire en sorte de mettre en place un système de personnalisation de l'interface pour que l'utilisateur puisse avoir un environnement de travail qui lui correspond au mieux.

IV. Annexes

IV.1 ANNEXE A : Annexes Docker

Installation des dépendances

```
apt-get update && apt-get install -y \  
unzip \  
git \  
libc-dev \  
libpq-dev \  
curl \  
&& docker-php-ext-install intl pdo_pgsql pgsql opcache \  
&& pecl install xdebug \  
&& docker-php-ext-enable xdebug
```

Configuration PHP en mode développement

```
mv "$PHP_INI_DIR/php.ini-development" "$PHP_INI_DIR/php.ini"
```

Installation de Composer

```
curl -sS https://getcomposer.org/installer | php -- --install-dir=/usr/local/bin  
--filename=composer
```

Installation de Symfony CLI

```
curl -sS https://get.symfony.com/cli/installer | bash \  
&& mv /root/.symfony5/bin/symfony /usr/local/bin/
```

Définition du répertoire de travail

```
WORKDIR /var/www/html
```

Copie du projet Symfony dans le conteneur

```
git config --global --add safe.directory /var/www/html  
git clone  
https://oauth2:G7AD2docjSwkfT2hVu_t@gricad-gitlab.univ-grenoble-alpes.fr/iut2-info-stud/  
2025-s4/sae-4a/team-15/carnet-stage-serveur-24-25.git .
```

Modification des permissions

```
chown -R www-data:www-data /var/www/html \  
&& chmod -R 755 /var/www/html  
echo "APP_ENV=dev" >> /var/www/html/.env && \  
echo "APP_SECRET=2a6fbfe22f3578d7dc9d64c36f2757a9">> /var/www/html/.env && \  

```

```

echo      "MESSENGER_TRANSPORT_DSN=doctrine://default?auto_setup=0"      >>
/var/www/html/.env && \
echo
"DATABASE_URL=\"postgresql://app-stages:app-stages@postgres_db:5432/app-stages?serv
erVersion=15&charset=utf8\"" >> /var/www/html/.env && \
echo      "CORS_ALLOW_ORIGIN='^https?:/(localhost|127\.\0\.\0\1)(:[0-9]+)?$'">>
/var/www/html/.env && \
echo      "JWT_SECRET_KEY=%kernel.project_dir%/config/jwt/private.pem"      >>
/var/www/html/.env && \
echo      "JWT_PUBLIC_KEY=%kernel.project_dir%/config/jwt/public.pem">>
/var/www/html/.env && \
echo "JWT_PASSPHRASE=e9b150b52547fdc546adc00ad015552c" >> /var/www/html/.env
# Installation des dépendances Composer
composer install --no-interaction --no-progress --prefer-dist

```

Scripts pour lancer les conteneurs

Script pour lancer le conteneur symfony à partir de l'image créée :

```

docker run -d --name $SYMFONY_CONTAINER \
  --network $NETWORK_NAME \
  --publish 8000:8000 \
  $SYMFONY_IMAGE

```

Script pour lancer le conteneur postgres_db à partir de l'image créée :

```

docker run -d --name $POSTGRES_CONTAINER \
  --network $NETWORK_NAME \
  -e POSTGRES_PASSWORD=app-stages \
  -e POSTGRES_DB=app-stages \
  $POSTGRES_IMAGE

```

IV. 2 ANNEXE B: Nouvelle structure de base de données

1. Table *candidature*

Colonne	Type	Description
<i>login</i>	INTEGER (PK FK → login)	Étudiant ayant postulé
<i>id_offre</i>	INTEGER (PK FK → id.offre)	Offre à laquelle l'étudiant postule
<i>type_acti on</i>	VARCHAR(255)	Action réalisée (ex: Envoi CV+LM)

date_action	TIMESTAMP	Date de l'action
etat	VARCHAR(255)	État (ex: Acceptée, Refusée, Attente)
descriptif	TEXT	Description de l'état

2. Table `compte_etudiant`

Colonne	Type	Description
login	VARCHAR(180)	Nom d'utilisateur
roles	JSON	Rôles attribués (admin, étudiant...)
password	VARCHAR(255)	Mot de passe
parcours	VARCHAR(1)	Type de parcours
derniere_connexion	TIMESTAMP (NULLABLE)	Dernière connexion
etat_recherche	VARCHAR(255)	État de la recherche de stage (ex: À Commencer, Terminée)
descriptif_recherche	TEXT	Description de l'état
numero_ine	VARCHAR(11)	Numéro INE de l'étudiant
nom	VARCHAR(255)	Nom
prenom	VARCHAR(255)	Prénom
email	VARCHAR(255)	Adresse e-mail

3. Table `compte_enseignant`

Colonne	Type	Description
login	VARCHAR(180)	Nom d'utilisateur

password	VARCHAR(255)	Mot de passe
derniere_connexion	TIMESTAMP (NULLABLE)	Dernière connexion

4. Table entreprise

Colonne	Type	Description
siret	VARCHAR (PK 17)	Identifiant unique (Siret + numéro incrémenté)
raison_sociale	VARCHAR(255)	Titre de l'offre
ville	TEXT (NULLABLE)	Description du poste
pays	DATE	Date de publication

5. Table offre

Colonne	Type	Description
id_offre	VARCHAR (PK 17)	Identifiant unique (Siret + numéro incrémenté)
intitule	VARCHAR(255)	Titre de l'offre
descriptif	TEXT (NULLABLE)	Description du poste
date_depot	DATE	Date de publication
parcours	VARCHAR(1) (NULLABLE)	Type de parcours concerné
mots_cles	VARCHAR(255) (NULLABLE)	Mots-clés associés
url_piece_jointe	VARCHAR(2000) (NULLABLE)	Lien vers le fichier de l'offre
etat	VARCHAR(255)	État de l'offre (ex: Disponible, Expirée)

descriptif_etat

TEXT

Description de l'état

6. Table offre_etu

Colonne	Type	Description
login	INTEGER (PK FK → login)	Étudiant ayant consulté l'offre
id_offre	INTEGER (PK FK → id.offre)	Offre consultée
estRetenu	BOOLEAN	Si l'offre est retenu ou non

IV. 3 ANNEXE C : Interfaces android studio

Image MainActivity



Image listOffres

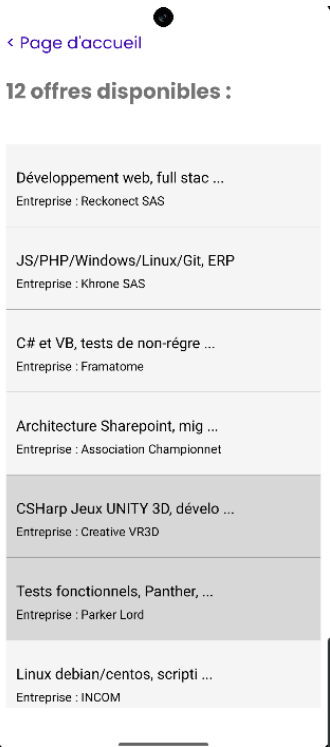


Image EditCandidature

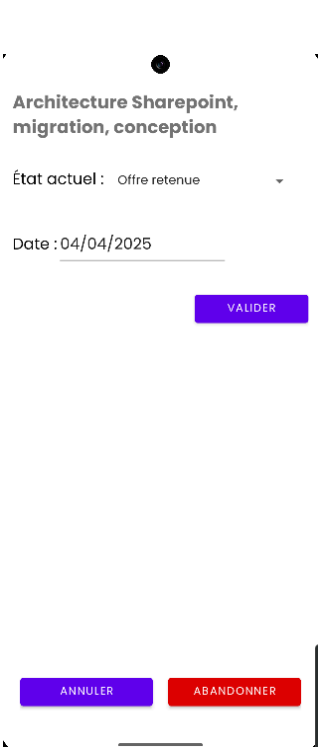


Image LoginActivity

Mon Carnet de Suivi de
Recherche de Stage

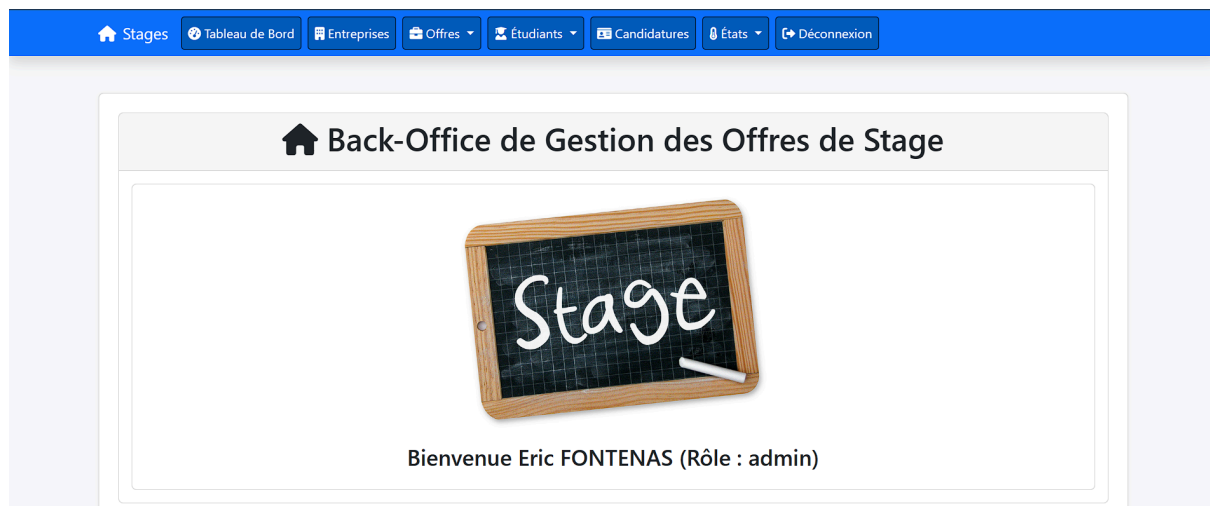
borealea

S'AUTHTIFIER

IUT2A
Université Grenoble Alpes
Département
INFO

1 2 3 4 5 6 7 8 9 0
q w e r t y u i o p
a s d f g h j k l
z x c v b n m
?123 Identifiant ou mot de passe incorrect. ✓

IV.4 ANNEXE D : Interfaces WEB Symfony



Nouvelle interface Admin.

modification de la navbar.

Stages

Tableau de Bord

Entreprises

Offres

Étudiants

Candidatures

États

Déconnexion

Entreprises

Rechercher :

ID	Raison Sociale	Ville	Pays	Actions
056			France	 
005	Association Championnet	Sallanches	France	 
006	Creative VR3D	Cannes	France	 
011	ELYXOFT	Gillonnay	France	 
004	Framatome	Ugine	France	 
054	Gillonnay	Gillonnay	France	 
055	Gillonnay	Gillonnay	France	 
013	HARDIS	Seyssinet-Pariset	France	 
019	INCOM	Gieres		 
008	INCOM	Gieres	France	 

Affichage de 1 à 10 sur 20 entrées

Précédente12Suivante

modification de l’interface d’affichage des entreprises.

Stages

Tableau de Bord

Entreprises

Offres

Étudiants

Candidatures

États

Déconnexion

Étudiants

Rechercher :

ID	Numéro INE	Nom	Prénom	Email	Actions
005	4567890123E	BOREALE	Aurore	aurore.boreale@iut2.fr	 
003	2345678901C	ENFAITE	Mélusine	melusine.enfaite@iut2.fr	 
001	0123456789A	FONTENAS	Eric	eric.fontenas@univ-grenoble-alpes.fr	 
010	5678901234F	HAPLUTARD	Jérémy	jeremy.haplutard@iut2.fr	 
004	3456789012D	HERBIEN	Jean-Philippe	jean-philippe.herbien@iut2.fr	 
002	9876543210B	KHONNU	Alain	alain.khonnu@iut2.fr	 
011	6789012345T	LARAISON	Jean-Pierre	jean-pierre.laraison@iut2.fr	 

Affichage de 1 à 7 sur 7 entrées

Précédente1Suivante

modification de l’interface de l’affichage des etudiants

IV. 5 ANNEXE E : Références webographiques

- [1] ChatGPT OpenAI chat.com
- [2] Claude AI claude.ai
- [3] StackOverflow stackoverflow.com
- [4] DockerHub hub.docker.com